

Prolepsis: System Architecture & Closed-Loop Thermal Management

A Comprehensive Technical Textbook and Architectural Reference Manual

Design, Microarchitecture, Fixed-Point Control Theory, RTL Implementation, and FPGA Verification of an ISA-Agnostic Heterogeneous RISC-V Multicore SoC

by **Varada Govind Aakula**

Advanced Computer Architecture & VLSI Systems Group

Authoritative System Architecture Reference | Active RTL Workspace

Document Purpose & Pedagogical Scope: This text provides an exhaustive, mathematically rigorous, and microarchitecturally complete explanation of the **Prolepsis** heterogeneous multicore System-on-Chip (SoC). Prolepsis demonstrates an end-to-end, all-hardware closed-loop thermal management subsystem implemented in SystemVerilog. Unlike conventional reactive Dynamic Voltage and Frequency Scaling (DVFS) or crude threshold throttling, Prolepsis continually samples microarchitectural execution telemetry, predicts future temperature trajectories using a pipelined fixed-point Kalman estimator, detects execution workload phases (compute-bound vs. memory-bound), and executes coordinated thermal actions: clock-enable duty cycle modulation, dynamic bus Quality-of-Service (QoS) bandwidth adjustments, proactive task migration with hardware instruction cache pre-warming, and multi-stage idle power gating. This manual covers every equation, pipeline stage, cache geometry, bus arbitration state, finite state machine, and MMIO register in the active codebase.

Contents

1	Foundations of Multicore Thermal Dynamics and the Prolepsis Paradigm	3
1.1	Physical Heat Dissipation and Silicon Thermal Inertia	3
1.2	The Architectural Breakdown of Reactive Thermal Throttling	3
1.3	The Prolepsis Principle: Anticipatory Closed-Loop Hardware Control	3
1.4	SoC Specifications and Architectural Parameters	4
2	Heterogeneous RISC-V Microarchitecture and Datapath	4
2.1	Heterogeneous Core Topology: P-Cores vs. E-Core	4
2.2	Five-Stage In-Order Pipeline Architecture	4
2.2.1	Stage 1: Instruction Fetch (IF)	6
2.2.2	Stage 2: Instruction Decode (ID) and Register File	6
2.2.3	Stage 3: Execution (EX), ALU, and Branch Evaluation	6
2.2.4	Stage 4: Memory Access (MEM)	6
2.2.5	Stage 5: Writeback (WB)	7
2.3	Dynamic Branch Predictor Subsystem	7
2.4	Hazard Detection, Forwarding, and Pipeline Interlocks	7
2.5	CSR Unit, Traps, and Custom Telemetry Extensions	8
3	Memory Subsystem, Coherence, and Interconnect Fabric	8
3.1	Private L1 Instruction Cache Architecture	8
3.2	Private L1 Data Cache, Victim Buffer, and Stride Prefetcher	9
3.3	Multicore Hardware Snoop Broadcast Coherence	9

3.4	AXI4-Lite Protocol and Interconnect Topology	9
3.5	Quality-of-Service (QoS) Arbiter Architecture	10
4	Theoretical Formulation and RTL Thermal Engine	10
4.1	Synthesizable Three-Node Lumped RC Thermal Network	10
4.2	Fixed-Point Arithmetic Format: Q16.16 and Q8.8	11
4.3	Pipelined Kalman-Style Predictive Temperature Estimator	11
4.3.1	Division-Free Gain Shift Approximation	11
4.4	Workload Phase Detection and Hysteresis Noise Suppression	12
4.5	Two-Layer Hierarchical Controller and Thermal Modes	12
5	Proactive Task Migration, Cache Warming, and Power Gating	13
5.1	Cost-Benefit Migration Policy Formulation	13
5.2	Deterministic Hardware State Transfer Sequence	13
5.3	Cross-Core Instruction Cache Pre-Warmer	14
5.4	Hierarchical Power-Gating Controller	14
6	Memory Map, Memory-Mapped I/O, and Peripheral Architecture	14
6.1	Global Address Space Organization	14
6.2	Comprehensive MMIO Register Map Reference	15
6.3	Peripherals: Timer, UART, and Seven-Segment Driver	17
7	Verification Suite, Synthesis, and Physical FPGA Implementation	17
7.1	Automated Simulation Regression Matrix	17
7.2	FPGA Physical Implementation on AMD Xilinx UltraScale+	17
7.3	Clock-Enable Domain Safety and Glitch Elimination	17
8	Architectural Trade-Off Analysis and Comparative Evaluation	19
9	Exhaustive Technical Question and Answer Reference	19
10	Repository Manifest and Automation Quick Reference	21
10.1	Essential Source Files for Architectural Review	21
10.2	Primary Verification and Build Commands	21

1 Foundations of Multicore Thermal Dynamics and the Prolepsis Paradigm

1.1 Physical Heat Dissipation and Silicon Thermal Inertia

In modern complementary metal-oxide-semiconductor (CMOS) integrated circuits, power dissipation comprises two primary components: dynamic switching power (P_{dyn}) and static leakage power (P_{leak}):

$$P_{\text{total}} = P_{\text{dyn}} + P_{\text{leak}} = \alpha C_{\text{load}} V_{\text{dd}}^2 f_{\text{clk}} + I_{\text{leak}} V_{\text{dd}} \quad (1)$$

where α represents the switching activity factor, C_{load} is the lumped capacitive load, V_{dd} is the supply voltage, f_{clk} is the operational clock frequency, and I_{leak} is the subthreshold and gate leakage current (which increases exponentially with junction temperature).

When a processor core transitions from an idle state to an intensive arithmetic loop, dynamic power surges almost instantaneously (on the order of picoseconds). However, the physical temperature of the silicon die does not rise instantly. Silicon possesses finite **thermal capacitance** (C_{th}) and **thermal resistance** (R_{th}). Heat generated at the microscopic transistor junction must conduct through the bulk silicon, thermal interface material (TIM), copper heat spreader, and external heatsink into ambient air. The thermal response follows exponential dynamics governed by thermal time constants:

$$\tau_{\text{th}} = R_{\text{th}} \cdot C_{\text{th}} \quad (2)$$

Typical silicon junction time constants range from milliseconds to hundreds of milliseconds, while package time constants span seconds. This fundamental delay between power generation and thermal manifestation is termed **thermal inertia**.

1.2 The Architectural Breakdown of Reactive Thermal Throttling

Conventional Dynamic Thermal Management (DTM) policies rely on reactive threshold detection: when an on-die thermal diode or analog-to-digital converter (e.g., Xilinx XADC) registers that temperature T_{meas} has crossed a thermal threshold T_{warn} or T_{crit} , a software interrupt or hardware governor engages DVFS or clock gating.

The Reactive Control Dilemma:

1. Core executes intensive workload $\rightarrow$ Power spikes instantly to P_{peak} .
2. Junction heats gradually due to thermal capacitance τ_{th} .
3. T_{meas} crosses T_{warn} $\rightarrow$ Reactive controller engages throttling.
4. **Overshoot Phenomenon:** Because the underlying thermal mass (heat spreader and die bulk) is already charged with thermal energy, temperature continues to rise *after* power is reduced, overshooting T_{crit} .
5. Emergency thermal shutdown or aggressive clock chopping is triggered, causing catastrophic performance collapse.

Figure 1: The physical mechanism of reactive thermal overshoot and instability.

Furthermore, reactive controllers are **workload-agnostic**: they penalize all cores equally regardless of whether a core is actively generating switching heat or stalling on memory accesses. Slowing down a memory-bound core yields virtually zero power reduction while drastically increasing workload execution time and system latency.

1.3 The Prolepsis Principle: Anticipatory Closed-Loop Hardware Control

Prolepsis replaces reactive thresholds with a continuous, hardware-embedded predictive control loop. The hardware architecture is organized around four fundamental questions answered in real-time by dedicated digital logic:

1. **Observation:** *What is the current computational state and activity level of every core?* $\implies$ Captured via per-core hardware performance counters (retired instructions, active cycles, cache hits, cache misses).
2. **Prediction:** *What temperature will each core reach in the near future if current execution continues?* $\implies$ Calculated via a pipelined, division-free Kalman filter operating in Q16.16 fixed-point arithmetic.
3. **Characterization:** *Is the core performing compute-intensive arithmetic or stalling on external memory?* $\implies$ Evaluated via a dual Exponential Moving Average (EMA) phase detector with hysteresis noise rejection.
4. **Actuation:** *Which hardware control mechanism provides optimal thermal relief with minimal performance loss?* $\implies$ Arbitrated among clock-enable duty modulation, bus QoS re-weighting, proactive task migration with hardware cache warming, and idle power gating.

The Closed-Loop Prolepsis Architectural Pipeline:

Telemetry Monitoring $\rightarrow$ Kalman State Estimation $\rightarrow$ Workload Phase Classifier
 $\rightarrow$ Cost-Benefit Policy Engine
 $\rightarrow$ Coordinated Hardware Actuation $\rightarrow$ Closed Feedback

1.4 SoC Specifications and Architectural Parameters

2 Heterogeneous RISC-V Microarchitecture and Datapath

2.1 Heterogeneous Core Topology: P-Cores vs. E-Core

Prolepsis implements a heterogeneous multicore arrangement consisting of five cores designated Hart 0 through Hart 4:

- **Cores 0, 1, 2, 3 (Performance Cores / P-Cores):** Configured with higher default interconnect priority (IFetch QoS = 3, Data QoS = 4). Intended for primary multi-threaded compute workloads.
- **Core 4 (Efficiency Core / E-Core):** Configured with lower default interconnect priority (IFetch QoS = 1, Data QoS = 2). Serves as an energy-efficient destination for memory-bound tasks or a thermal relief vessel during task migration.

All five cores share the identical RV32IMAC datapath definition, ensuring binary portability and seamless register-state transfer during migration without ISA translation layers.

2.2 Five-Stage In-Order Pipeline Architecture

The core datapath (`rtl/core/RV32_Core.sv`) is structured as an in-order five-stage pipeline with synchronous pipeline registers isolating each stage:

IF (Fetch)	ID (Decode)	EX (Execute)	MEM (Memory)	WB (Writeback)
PC Generation	Regfile Read	ALU Arithmetic	D-Cache Access	Regfile Write
I-Cache Query	Immediate Gen	Multiplier / Div	MMIO Routing	CSR Commit
BHT / BTB / RAS	RVC Decompress	Branch Resolve	Atomic Siphon	Link PC (x_1)
Loop Buffer Check	Hazard Interlock	Forwarding Mux	Store Buffer	Migration Commit

Figure 2: Detailed five-stage RISC-V pipeline organization.

Subsystem	Architectural Parameter	Value / Implementation Details
Processor Cores	Architecture	5-Core Heterogeneous RISC-V RV32IMAC
	Topology	4 Performance Cores (P-Cores) + 1 Efficiency Core (E-Core)
	Pipeline Structure	5-Stage In-Order Single-Issue (IF → ID → EX → MEM → WB)
	Branch Predictor	64-entry BHT (2-bit) + 6-bit GHR + 64-entry BTB + 8-entry RAS
Memory Hierarchy	L1 Instruction Cache	Private 4 KB Direct-Mapped per core (256 lines × 16 bytes)
	L1 Data Cache	Private 4 KB Direct-Mapped, Write-Through, Write-Allocate
	Hardware Assists	2-Entry Fully Associative Victim Buffer + 4-Entry Stride Prefetcher
Interconnect	Bus Topology	Shared single-outstanding AXI4-Lite bus (11 Masters × 5 Slaves)
	Arbitration	3-bit Priority QoS Arbiter with 8-bit age starvation counter (200-cycle threshold)
Memory & I/O	Main SRAM	256 KB Synthesized Synchronous Block RAM (0x2000_0000)
	Peripherals	Boot ROM (4 KB), Timer (64-bit), UART (115.2k), 7-Seg Driver, MAC Unit
Thermal Engine	Thermal Plant	Synthesizable 3-Node Lumped RC Differential Model (T_j, T_s, T_p)
	Arithmetic Format	Signed 32-bit Q16.16 Fixed-Point ($2^{16} = 65536$ scaling)
	State Estimator	4-Stage Pipelined Division-Free Kalman Predictor with Shift Gain
	Phase Classifier	4096-Cycle Window, Dual EMA ($\alpha = 1/8, 1/64$), 8-Sample Hysteresis
Physical Target	FPGA Platform	AMD Xilinx UltraScale+ XCU250 (xcu250-figd2104-2L-e)
	Clock Frequency	98.0 MHz ($T_{\text{clk}} = 10.204$ ns, registered clock-enable domain)
	Power & Slack	Total On-Chip Power: 3.720 W; WNS: +0.057 ns, WHS: +0.011 ns

Table 1: Comprehensive Architectural and Implementation Parameters of Prolepsis SoC.

2.2.1 Stage 1: Instruction Fetch (IF)

The IF stage maintains the 32-bit Program Counter (`pc`). The next PC is determined by a priority multiplexer:

$$PC_{\text{next}} = \begin{cases} PC_{\text{trap_target}} & \text{if trap, interrupt, or MRET asserted} \\ PC_{\text{branch_correct}} & \text{if branch misprediction resolved in EX} \\ PC_{\text{migration}} & \text{if migration PC restore asserted} \\ PC_{\text{ras}} & \text{if predicted return instruction} \\ PC_{\text{btb}} & \text{if predicted taken branch with BTB hit} \\ PC + 4 \text{ (or } PC + 2) & \text{standard sequential progression} \end{cases} \quad (3)$$

The PC is presented to the private L1 instruction cache. If the loop buffer is active, instructions are served directly from the loop buffer register array without triggering cache access or bus power.

2.2.2 Stage 2: Instruction Decode (ID) and Register File

The ID stage decodes 32-bit RISC-V instructions and manages architectural register reads:

- **Register File (`Register_File.sv`):** Contains 32 32-bit general-purpose registers ($x_0 \dots x_{31}$). x_0 is hardwired to zero. Implements two asynchronous read ports (`rdata1`, `rdata2`) and one synchronous write port. A dedicated migration write port overrides normal writeback during state restoration.
- **Immediate Generator (`Imm_Gen.sv`):** Extracts and sign-extends immediates across all standard formats: I-type (12-bit arithmetic/loads), S-type (12-bit stores), B-type (13-bit conditional branches), U-type (20-bit upper immediates), and J-type (21-bit unconditional jumps).
- **RVC Decompressor (`RVC-Decompressor.sv`):** Identifies 16-bit compressed instructions (when bits `[1:0] != 2'b11`) across Quadrants 0, 1, and 2, expanding compressed loads, stores, additions, branches, and register moves into standard 32-bit base instructions in real time with zero pipeline stalls.
- **Loop Buffer (`Loop_Buffer.sv`):** Captures short backward loops (up to 16 instructions). When a backward branch is detected, the loop body is cached; subsequent iterations stream from the loop buffer, disabling IF stage cache reads and saving clock power.

2.2.3 Stage 3: Execution (EX), ALU, and Branch Evaluation

The EX stage performs arithmetic operations, evaluates branch conditions, and calculates memory effective addresses:

- **Pipelined ALU (`Pipelined_ALU.sv`):** Executes 32-bit arithmetic operations: addition, subtraction, set-less-than (signed/unsigned), logical AND, OR, XOR, and barrel shifts (SLL, SRL, SRA).
- **Multiplication and Division Unit (`RV32M`):** Hardware multiplier utilizes FPGA DSP48 slices. Hardware integer division/remainder uses a multi-cycle sequential restoring divider with `busy` and `done` handshaking signals to stall the pipeline during calculation.
- **Branch Unit (`Branch_Unit.sv`):** Compares operands for conditional branches (BEQ, BNE, BLT, BGE, BLTU, BGEU). If the branch outcome diverges from the branch predictor's guess, a misprediction signal is asserted, flushing the IF and ID stages.

2.2.4 Stage 4: Memory Access (MEM)

The MEM stage interfaces with the private L1 data cache:

- Generates 4-bit byte write strobes (**wstrb**) based on instruction width: byte (**SB**), halfword (**SH**), or word (**SW**).
- Formats read data via sign extension (**LB**, **LH**, **LW**) or zero extension (**LBU**, **LHU**).
- Manages atomic memory operations (**LR.W**, **SC.W**) by establishing and verifying an internal hardware reservation register.

2.2.5 Stage 5: Writeback (WB)

The WB stage commits the final result to the register file. A multiplexer selects among: (1) ALU output, (2) D-Cache load data, (3) Link PC ($PC + 4$ or $PC + 2$ for subroutine calls), or (4) CSR read data.

2.3 Dynamic Branch Predictor Subsystem

The branch prediction engine (`rtl/core/Branch_Predictor.sv`) coordinates four distinct hardware structures to maximize instruction throughput:

1. **64-Entry Branch History Table (BHT)**: Array of 2-bit saturating up-down counters representing four prediction states: ‘Strongly Not Taken’ (00), ‘Weakly Not Taken’ (01), ‘Weakly Taken’ (10), and ‘Strongly Taken’ (11).
2. **6-Bit Global History Register (GHR)**: Shift register recording the outcome (1 = taken, 0 = not taken) of the last 6 conditional branches across the program.
3. **XOR Index Hashing (Gshare)**: The BHT is indexed via an XOR hash of lower PC bits and the GHR:

$$\text{Index}_{\text{BHT}} = \text{PC}[7 : 2] \oplus \text{GHR}[5 : 0] \quad (4)$$

This eliminates branch aliasing between different code paths sharing identical PC offsets.

4. **64-Entry Branch Target Buffer (BTB)**: Direct-mapped cache storing full 32-bit target PCs with 20-bit tag comparison.
5. **8-Entry Return Address Stack (RAS)**: Dedicated circular LIFO stack. Instructions with $rd = x_1$ or x_5 (calls) push $PC + 4$ onto the RAS; instructions with $rs1 = x_1$ or x_5 and $rd = x_0$ (returns) pop the target PC directly, achieving near 100% prediction accuracy on subroutine returns.

2.4 Hazard Detection, Forwarding, and Pipeline Interlocks

The hazard unit (`rtl/core/Hazard_Unit.sv`) guarantees correctness without unnecessary performance degradation:

- **EX-to-EX Forwarding**: When an instruction in EX requires a register currently being computed by the instruction in MEM, the result is bypassed directly from the EX/MEM pipeline register to the ALU operand input.
- **MEM-to-EX Forwarding**: When an instruction in EX requires a register being committed in WB, data is bypassed from the MEM/WB register.
- **Load-Use Hazard Interlock**: Because data from a load instruction is not available until the end of the MEM stage, an instruction in ID that depends on the load result cannot execute immediately. The hazard unit detects this condition:

$$\text{Stall}_{\text{load_use}} = \text{RegWrite}_{\text{EX}} \wedge (\text{MemRead}_{\text{EX}}) \wedge ((rd_{\text{EX}} == rs1_{\text{ID}}) \vee (rd_{\text{EX}} == rs2_{\text{ID}})) \quad (5)$$

When asserted, the IF and ID pipeline registers freeze for one cycle, and a bubble (NOP) is inserted into EX.

- **Bus and Cache Stalls**: When an L1 cache miss occurs or the AXI bus is waiting for arbitration grant, the entire pipeline freezes synchronously.

2.5 CSR Unit, Traps, and Custom Telemetry Extensions

The Control and Status Register unit (`rtl/core/CSR_Unit.sv`) implements standard RISC-V Privileged Architecture machine-mode registers:

CSR Name	Address	Architectural Function
<code>mstatus</code>	0x300	Machine status register (global interrupt enable MIE, prior enable MPIE)
<code>misa</code>	0x301	Machine ISA features (encodes RV32IMAC capabilities)
<code>mie</code>	0x304	Machine interrupt enable register (timer, external, and thermal IRQ bits)
<code>mtvec</code>	0x305	Machine trap-handler base address vector
<code>mscratch</code>	0x340	Scratch register for trap handler context save
<code>mepc</code>	0x341	Machine exception program counter (PC saved on trap entry)
<code>mcause</code>	0x342	Machine trap cause (identifies interrupt or exception code)
<code>mtval</code>	0x343	Machine bad address or faulting instruction representation
<code>mip</code>	0x344	Machine interrupt pending register
<code>mcycle/minstret</code>	0xB00/0xB02	64-bit cycle and retired instruction hardware counters
<code>mhartid</code>	0xF14	Hardware thread identifier (0 to 4)

Additionally, the core exposes custom migration CSR read/write access slots ('0x0' to '0xF') through which the migration controller rapidly checkpoints and restores core-local machine state during task transfers.

3 Memory Subsystem, Coherence, and Interconnect Fabric

3.1 Private L1 Instruction Cache Architecture

The L1 Instruction Cache (`rtl/cache/Instruction_Cache.sv`) is a private 4 KB direct-mapped SRAM cache designed for single-cycle access:

- **Organization:** 256 cache lines, each holding 16 bytes (four 32-bit words). Total capacity = $256 \times 16 = 4096$ bytes.
- **Bitfield Mapping:**

Tag Field: [31:12]	Index Field: [11:4]	Word Select: [3:2]	Byte Offset: [1:0]
20 bits	8 bits (256 lines)	2 bits (4 words/line)	2 bits (4 bytes/word)

- **Finite State Machine (FSM):** Operates across five states:
 - **IDLE:** Monitors CPU fetch requests. On hit, delivers data in 1 cycle. On miss, latches missing address and transitions to **FILL**.
 - **FILL:** Issues four consecutive word-read requests across the AXI bus wrapper.
 - **WAIT:** Collects incoming bus read beats, assembling the 128-bit cache line.
 - **COMPLETE:** Writes the assembled line into tag and data SRAM arrays, sets the valid bit, and delivers the critical requested word to the core.
 - **PREFETCH:** Issues background next-line fetch if sequential access threshold is met.
- **Inspection Port for Cache Warming:** Exposes an inspection address port (`inspect_index`) allowing the external cache warmer to query valid bits and tag entries without stalling the core pipeline.

3.2 Private L1 Data Cache, Victim Buffer, and Stride Prefetcher

The L1 Data Cache (`rtl/cache/Data_Cache.sv`) is a 4 KB direct-mapped, write-through, write-allocate cache paired with hardware acceleration structures:

- **Write-Through / Write-Allocate Policy:** On a store hit or miss, data is written to the L1 cache line and simultaneously dispatched to main memory via a single-entry store buffer. Because main memory is always up to date, lines never become “dirty”, completely eliminating the latency and complexity of writeback eviction buffers.
- **Single-Entry Store Buffer:** Buffers outgoing stores to decouple core execution from bus write latency. If a subsequent load targets the address currently residing in the store buffer, data is forwarded directly from the buffer without accessing memory.
- **2-Entry Fully Associative Victim Buffer (`Victim_Buffer.sv`):** Whenever an incoming line evicts an existing valid line from the direct-mapped L1 cache, the evicted line is pushed into a 2-entry FIFO victim buffer. On a subsequent L1 miss, the victim buffer is searched in parallel with the main tag lookup; a victim hit swaps the requested line back into L1 within a single clock cycle, neutralizing conflict misses.
- **4-Entry Stride Prefetcher (`Stride_Prefetcher.sv`):** Tracks historical memory reference strides:

$$\text{Stride} = \text{Addr}[k] - \text{Addr}[k - 1] \quad (6)$$

When two consecutive accesses exhibit identical strides with high state confidence, the prefetcher issues speculative line requests to the bus arbiter during idle cycles, hiding memory latency for sequential array processing.

3.3 Multicore Hardware Snoop Broadcast Coherence

Data coherence across all five private L1 data caches is enforced via a hardware snoop bus coordinated by the top-level module (`rtl/Orionrv.sv`):

1. Whenever Core i executes a write-through store, the bus interconnect registers the store address $\text{Addr}_{\text{store}}$ and writing source core ID.
2. The top level broadcasts $\text{Addr}_{\text{store}}$ and SourceID to all other cores ($j \neq i$).
3. Each remote data cache compares $\text{Addr}_{\text{store}}[31 : 4]$ against its local cache tags at index $\text{Addr}_{\text{store}}[11 : 4]$.
4. If a match is detected, the remote cache line’s valid bit is cleared (**invalidation**).
5. Concurrently, if the broadcast store address matches an active Load-Reserved reservation tag, the reservation valid bit is cleared, ensuring remote Store-Conditional operations fail appropriately.

3.4 AXI4-Lite Protocol and Interconnect Topology

The interconnect architecture connects 11 bus masters to 5 memory-mapped slaves:

- **11 Masters:** Masters 0–4 (I-Caches for Cores 0–4), Masters 5–9 (D-Caches for Cores 0–4), Master 10 (MAC Accelerator).
- **5 Slaves:** Slave 0 (Boot ROM), Slave 1 (RAM), Slave 2 (VGA Stub), Slave 3 (MMIO), Slave 4 (MAC).
- **Single-Outstanding Atomic Operation:** To minimize FPGA multiplexer fanout and guarantee deadlock freedom, the interconnect processes one atomic transaction at a time. The master wrapper couples write address (**AW**) and write data (**W**) channels, guaranteeing single-beat completion.

3.5 Quality-of-Service (QoS) Arbiter Architecture

The QoS Arbiter (`rtl/bus/qos_arbiter.sv`) implements a multi-stage pipelined priority scheduler:

- **3-Bit QoS Priority Levels:** Higher values receive preferential bus access:

QoS Level	Numerical Value	Default Master Assignment
QOS_LOW	3'd0	Background tasks, lowest priority
QOS_ECORE_IFETCH	3'd1	Core 4 (E-Core) instruction fetch
QOS_ECORE_DATA	3'd2	Core 4 (E-Core) data memory access
QOS_PCORE_IFETCH	3'd3	Cores 0-3 (P-Cores) instruction fetch
QOS_PCORE_DATA	3'd4	Cores 0-3 (P-Cores) data memory access
QOS_MAC_ACCEL	3'd5	Hardware MAC accelerator master
QOS_DMA	3'd6	High-priority DMA channel
QOS_CRITICAL	3'd7	Thermal emergency or critical memory sync

- **Anti-Starvation Age Counters:** Every master possesses an 8-bit age counter. If a master asserts a request but is denied grant for 200 consecutive clock cycles (`AGE_BOOST_THRESH`), its effective priority is boosted with a high-order bit, elevating it above all standard traffic.
- **Balanced Max-Reduction Tree:** Uses a 6-stage binary comparison tree (`max_stage0` to `max_stage5`) to identify the highest effective QoS across all 11 masters within a registered pipeline stage, eliminating deep combinational comparator chains.
- **Round-Robin Tie-Breaking:** Equal-priority candidates are resolved using a rotating round-robin pointer (`rr_ptr`).
- **Atomic Lock Support:** Masters executing atomic read-modify-write sequences assert `lock`, retaining grant across multi-cycle operations.

4 Theoretical Formulation and RTL Thermal Engine

4.1 Synthesizable Three-Node Lumped RC Thermal Network

Heat conduction through the physical package is modeled in hardware using a three-node lumped thermal circuit representing the transistor junction (T_j), integrated heat spreader (T_s), and package envelope (T_p):

The Three-Node Lumped Thermal Differential Equations:

Node 1 (Junction):

$$C_j \frac{dT_j}{dt} = P_{\text{dynamic}} - \frac{T_j - T_s}{R_{js}} \quad (7)$$

Node 2 (Heat Spreader):

$$C_s \frac{dT_s}{dt} = \frac{T_j - T_s}{R_{js}} - \frac{T_s - T_p}{R_{sp}} \quad (8)$$

Node 3 (Package / Ambient):

$$C_p \frac{dT_p}{dt} = \frac{T_s - T_p}{R_{sp}} - \frac{T_p - T_{\text{ambient}}}{R_{pa}} \quad (9)$$

Figure 3: Continuous differential thermal model implemented in synthesizable RTL.

In the discrete digital domain, continuous derivatives are integrated via forward Euler steps with sampling period Δt :

$$T_j[k+1] = T_j[k] + \frac{\Delta t}{C_j} \left(P_{\text{dyn}}[k] - \frac{T_j[k] - T_s[k]}{R_{js}} \right) \quad (10)$$

In `Thermal_Sensor.sv`, parameters $\frac{\Delta t}{C_1}, \frac{\Delta t}{C_2}, \frac{\Delta t}{C_3}$ and $\frac{1}{R_{12}}, \frac{1}{R_{23}}, \frac{1}{R_{3a}}$ are parameterized in Q8.8 fixed-point, while temperatures are represented in Q16.16. A 12-bit Linear Feedback Shift Register (LFSR) injects configurable Gaussian-approximated sensor noise and static PVT bias.

4.2 Fixed-Point Arithmetic Format: Q16.16 and Q8.8

All arithmetic in the Prolepsis thermal engine operates in deterministic fixed-point notation:

- **Q16.16 Representation:** 32-bit signed integer where the upper 16 bits represent the integer part and the lower 16 bits represent the fractional part:

$$\text{Real Value} = \frac{\text{Stored Integer}}{2^{16}} = \frac{\text{Stored Integer}}{65536} \quad (11)$$

- $25.0^\circ\text{C} = 25 \times 65536 = 1638400 = 0x0019_0000$
- $85.0^\circ\text{C} = 85 \times 65536 = 5570560 = 0x0055_0000$
- $87.0^\circ\text{C} = 87 \times 65536 = 5701632 = 0x0057_0000$
- $90.0^\circ\text{C} = 90 \times 65536 = 5898240 = 0x005A_0000$

- **Fixed-Point Multiplication Normalization:** When two Q16.16 numbers are multiplied, the product produces a Q32.32 format (64 bits). Extracting bits [47:16] restores the standard Q16.16 alignment:

$$\text{Product}_{\text{Q16.16}} = (\text{OpA}_{\text{Q16.16}} \times \text{OpB}_{\text{Q16.16}}) \gg 16 \quad (12)$$

4.3 Pipelined Kalman-Style Predictive Temperature Estimator

The Kalman Predictor (`rtl/thermal/Kalman_Predictor.sv`) predicts future thermal state using a pipelined, four-stage discrete linear filter:

The Pipelined Kalman Hardware State Estimator:

Stage 1: Multiplier Array (DSP Mapping)

$$\begin{aligned} \text{term}_1 &= F \cdot T_{\text{est}}[k-1] & \text{term}_2 &= B \cdot P_{\text{power}}[k] \\ \text{term}_3 &= G \cdot T_{\text{amb}} & \text{term}_4 &= F \cdot P_{\text{cov}}[k-1] \end{aligned}$$

Stage 2: Prediction Summation and Covariance Projection

$$\begin{aligned} T_{\text{pred}}[k] &= \text{term}_1[47:16] + \text{term}_2[47:16] + \text{term}_3[47:16] \\ P_{\text{pred}}[k] &= (\text{term}_4[47:16] \cdot F) \gg 16 + Q \end{aligned}$$

Stage 3: Residual Innovation and Division-Free Gain Computation

$$\begin{aligned} y[k] &= T_{\text{sensor}}[k] - T_{\text{pred}}[k] \quad (\text{Innovation Residual}) \\ S[k] &= P_{\text{pred}}[k] + R \quad (\text{Innovation Covariance}) \end{aligned}$$

Stage 4: State Update and Uncertainty Correction

$$\begin{aligned} T_{\text{est}}[k] &= T_{\text{pred}}[k] + (y[k] \gg \text{shift}) \\ P_{\text{est}}[k] &= P_{\text{pred}}[k] - (P_{\text{pred}}[k] \gg \text{shift}) \end{aligned}$$

Figure 4: Mathematical pipeline structure of the hardware Kalman temperature predictor.

4.3.1 Division-Free Gain Shift Approximation

Standard Kalman filters require matrix division to compute the optimal gain $K = \frac{P_{\text{pred}}}{P_{\text{pred}} + R}$. In FPGA logic, a 32-bit divider introduces a severe timing bottleneck (120 ns critical path). Prolepsis replaces

division with a power-of-two shift gain comparator:

$$\text{Shift Amount } s = \begin{cases} 0 & \text{if } P_{\text{pred}} \geq S \quad (K \approx 1.0) \\ 1 & \text{if } P_{\text{pred}} \geq (S \gg 1) \quad (K \approx 0.5) \\ 2 & \text{if } P_{\text{pred}} \geq (S \gg 2) \quad (K \approx 0.25) \\ 3 & \text{if } P_{\text{pred}} \geq (S \gg 3) \quad (K \approx 0.125) \\ 4 & \text{if } P_{\text{pred}} \geq (S \gg 4) \quad (K \approx 0.0625) \\ 5 & \text{otherwise} \quad (K \approx 0.03125) \end{cases} \quad (13)$$

Applying the gain reduces to an arithmetic right shift ($y \gg s$), allowing the predictor to close timing at 98 MHz with zero DSP divider stalls.

4.4 Workload Phase Detection and Hysteresis Noise Suppression

The Phase Detector (`rtl/thermal/Phase_Detector.sv`) samples performance counters over a 4096-cycle window:

- **Shift-Based Ratio Classifiers (Zero Dividers):**

$$\text{High IPC} \iff \Delta_{\text{minstret}} > (\Delta_{\text{mcycle}} \gg 1) \quad (\text{IPC} > 0.5) \quad (14)$$

$$\text{Low IPC} \iff \Delta_{\text{minstret}} < (\Delta_{\text{mcycle}} \gg 3) \quad (\text{IPC} < 0.125) \quad (15)$$

$$\text{Idle IPC} \iff \Delta_{\text{minstret}} < (\Delta_{\text{mcycle}} \gg 6) \quad (\text{IPC} < 0.016) \quad (16)$$

$$\text{High Miss Rate} \iff \text{Miss} > (\text{Hit} + \text{Miss}) \gg 2 \quad (> 25\%) \quad (17)$$

$$\text{Low Miss Rate} \iff \text{Miss} < (\text{Hit} + \text{Miss}) \gg 4 \quad (< 6.25\%) \quad (18)$$

- **Dual Exponential Moving Average Smoothing:**

$$\text{EMA}_{\text{short}}[k] = \text{EMA}_{\text{short}}[k-1] + \frac{\text{Sample} - \text{EMA}_{\text{short}}[k-1]}{8} \quad (\alpha = 1/8) \quad (19)$$

$$\text{EMA}_{\text{long}}[k] = \text{EMA}_{\text{long}}[k-1] + \frac{\text{Sample} - \text{EMA}_{\text{long}}[k-1]}{64} \quad (\alpha = 1/64) \quad (20)$$

- **Phase State Assignments:**

- **Phase 0 (COMPUTE):** High IPC $\wedge$ Low Miss Rate $\wedge$ IPC Trend Up. Generates high heat; priority candidate for throttling and migration.
- **Phase 1 (MEMORY):** Low IPC $\wedge$ High Miss Rate. Stalling on memory; low thermal output; throttling is suppressed.
- **Phase 2 (BALANCED):** Intermediate execution properties.
- **Phase 3 (IDLE):** Near-zero activity. Candidate for power gating.

- **8-Sample Hysteresis Filter:** A candidate phase must persist continuously across 8 sampling windows (32,768 clock cycles) before changing the active output phase, eliminating high-frequency phase chatter.

4.5 Two-Layer Hierarchical Controller and Thermal Modes

The Thermal Controller (`rtl/thermal/Thermal_Controller.sv`) coordinates per-core actuation through a two-layer hierarchy:

1. **Layer 1: Hard Hardware Safety Governor (Independent & Combinational):**

- If $T_{\text{sensor}} > T_{\text{cap}}$ (87°C): Overrides all policies and forces a 25% clock duty cycle.
- If $T_{\text{sensor}} > T_{\text{crit}}$ (90°C): Immediately halts the clock (0% duty) and asserts a non-maskable thermal interrupt (`thermal_irq`).

2. **Layer 2: Adaptive Proactive Control (Selectable via Modes 0 to 3):**

Mode	Name	Input Trigger	Hardware Actuation and Duty Cycle Rules
0	Baseline	BTNC	Unthrottled 100% duty cycle. Reference baseline for all benchmark comparisons.
1	Reactive	BTNU	Reacts to T_{sensor} . If $T_{\text{sensor}} > 85^\circ\text{C}$, applies 50% clock duty cycle.
2	Predictive	BTNL	Reacts to T_{pred} . If $T_{\text{pred}} > 85^\circ\text{C}$, applies 75% clock duty cycle <i>before</i> temperature breach.
3	Full System	BTNR	Integrates prediction, phase detection, migration, warming, QoS, and power gating. If $T_{\text{pred}} > 85^\circ\text{C}$: Compute phase $\implies$ 25% duty; Memory phase $\implies$ 75% duty; Balanced phase $\implies$ 50% duty.

Table 2: Detailed Operational Specification of Prolepsis Thermal Modes.

5 Proactive Task Migration, Cache Warming, and Power Gating

5.1 Cost-Benefit Migration Policy Formulation

The Migration Policy Engine (`rtl/thermal/Migration_Policy.sv`) continuously evaluates whether migrating a running task will produce a net thermal-performance benefit:

- **Source Core Selection:** The source core is chosen as the hottest predicted core:

$$\text{Source Core } s = \arg \max_{i \in [0,4]} T_{\text{pred}}[i] \quad (21)$$

- **Destination Core Selection:** The destination core is chosen as the core with maximum available thermal headroom:

$$\text{Headroom}_j = \max(0, T_{\text{crit}} - T_{\text{pred}}[j]), \quad \text{Destination Core } d = \arg \max_{j \neq s} \text{Headroom}_j \quad (22)$$

- **Projected Benefit Score:**

$$\text{Benefit Score} = \left(\frac{T_{\text{pred}}[s] - T_{\text{pred}}[d]}{2^{12}} \right) + \text{Bonus}(\Phi_s) \quad (23)$$

where $\text{Bonus}(\Phi_s) = 64$ if the source core recently transitioned to compute-bound (Φ_{compute}).

- **Projected Cost in Cycles:**

$$\text{Cost Cycles} = C_{\text{base}} + C_{\text{warm}} + C_{\text{miss}} = 40 + \text{WarmCycles} + \left(\frac{\text{ICacheMiss}_s}{4} \right) \quad (24)$$

- **Trigger Condition:** Migration request (`migrate_req`) asserts if and only if:

$$(\text{Benefit} > \text{Cost}) \wedge (\neg \text{AllCoresHot}) \wedge (\neg \text{DualHot} \vee \text{ImbalanceOK}) \wedge (\text{CooldownCtr} == 0) \wedge (\neg \text{Busy}) \quad (25)$$

- **Cooldown Guard:** Once migration fires, a 10,000-cycle cooldown counter prevents thrashing. A 16-entry 64-bit circular ring buffer records historical migration events with cycle timestamps.

5.2 Deterministic Hardware State Transfer Sequence

The Migration Controller (`rtl/thermal/Migration_Controller.sv`) executes a 9-state finite state machine guaranteeing complete architectural state integrity:

State Name	Duration (Cycles)	Hardware Action and Signal Progression
ST_IDLE (0)	Variable	Awaits start pulse or power-gate request.
ST_PAUSE (1)	1 cycle	Asserts pause_mask for source and destination cores; freezes pipelines.
ST_READ_GPR (2)	31 cycles	Sets read index from x_1 to x_{31} ; captures source register data.
ST_WRITE_GPR (3)	31 cycles	Drives destination migration write port (mig_we); writes $x_1 \dots x_{31}$.
ST_READ_CSR (4)	16 cycles	Reads 16 machine CSR slots (mstatus , mepc , mtvec , telemetry).
ST_WRITE_CSR (5)	16 cycles	Writes 16 CSR slots into destination core CSR unit.
ST_WARM (6)	≤ 256 cycles	Triggers Cache Warmer; waits for cache_warm_done handshake.
ST_LOAD_PC (7)	1 cycle	Forces source checkpoint PC into destination core (pc_load_en).
ST_DONE (8)	1 cycle	Resumes destination core; latches source into halt_mask ; increments count.

Figure 5: The 9-state deterministic task migration sequence.

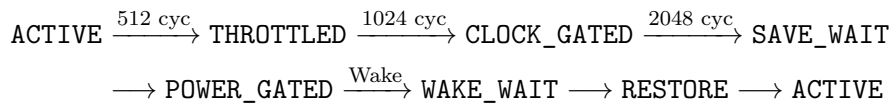
5.3 Cross-Core Instruction Cache Pre-Warmer

To eliminate the severe cold-start penalty after execution migrates, the Cache Warmer (`rtl/thermal/Cache_Warmer.sv`) acts during the migration pause window:

- Sniffs the source core’s L1 instruction cache valid bits and tags beginning from the checkpointed PC line index.
- Emits up to 16 line-aligned prefetch transactions (`prefetch_addr = inspect_tag, inspect_index, 4'b0000`) into the destination I-cache.
- Operates within a strict bounded window of 256 clock cycles, pre-populating working loops and eliminating cold cache stall bursts.

5.4 Hierarchical Power-Gating Controller

The Power Gate Controller (`rtl/thermal/Power_Gate_Controller.sv`) automatically stages idle cores into low-power states:



- **Idle Detection:** Increments a 12-bit idle counter when a core remains in Phase 3 (IDLE) and is not targeted by migration.
- **State Retention:** Upon reaching 2048 idle cycles, asserts **pg_save_req**. The migration controller saves all GPRs, CSRs, and PC into internal retention RAM before isolating the clock domain.
- **Deterministic Wake:** Upon task assignment, wakes the domain in 16 cycles, restores registers from retention RAM, and resumes execution seamlessly.

6 Memory Map, Memory-Mapped I/O, and Peripheral Architecture

6.1 Global Address Space Organization

The 32-bit physical address map is partitioned into distinct functional regions:

Slave Region	Base Address	Address Mask	Size	Functional Role
Boot ROM	0x0000_0000	0xFFFF_F000	4 KB	Non-volatile boot firmware loaded from <code>boot.hex</code> .
Main SRAM	0x2000_0000	0xFF80_0000	256 KB	High-speed on-chip data and program block RAM.
VGA Endpoint	0x4000_0000	0xFFFF_0000	64 KB	Framebuffer endpoint and display timing registers.
MMIO Control	0x9000_0000	0xFFFF_0000	64 KB	SoC hardware control registers, UART, and timer.
MAC Accelerator	0xA000_0000	0xFFFF_FF00	256 B	Dedicated Multiply-Accumulate compute engine.

Table 3: Global SoC Physical Address Map Partitioning.

6.2 Comprehensive MMIO Register Map Reference

The MMIO space at base 0x9000_0000 exposes complete runtime telemetry and control:

MMIO Offset	Register Symbol	Bitfield Definitions and Architectural Role
0x00	REG_LEDS	bits[7:0]: Board LEDs (thermal warning, active mode, QoS status).
0x04	REG_BUTTONS	bits[3:0]: Debounced push buttons (BTNC, BTNU, BTNL, BTNR).
0x08	REG_UART_TX	bits[7:0]: UART transmit FIFO write register.
0x0C	REG_UART_STATUS	bit[0]: TX busy, bit[1]: TX full, bit[2]: RX ready.
0x10	REG_MTIME_LO	bits[31:0]: Lower 32 bits of 64-bit real-time clock counter.
0x14	REG_MTIME_HI	bits[31:0]: Upper 32 bits of 64-bit real-time clock counter.
0x18	REG_MTIMECMP_LO	bits[31:0]: Lower 32 bits of timer compare (triggers timer interrupt).
0x1C	REG_MTIMECMP_HI	bits[31:0]: Upper 32 bits of timer compare register.
0x20	REG_TEMP_FUSED	bits[31:0]: Fused temperature of switch-selected core (Q16.16).
0x24	REG_TEMP_EST	bits[31:0]: Kalman estimated temperature of selected core (Q16.16).

MMIO Offset	Register Symbol	Bitfield Definitions and Architectural Role
0x28	REG_TEMP_PRED	bits[31:0]: Kalman predicted temperature of selected core (Q16.16).
0x2C	REG_THERMAL_STATE	bits[1:0]: State (0=Normal, 1=Warning, 2=Critical).
0x30	REG_PHASE	bits[1:0]: Workload phase (0=Compute, 1=Memory, 2=Balanced, 3=Idle).
0x34	REG_CORE_EN	bits[4:0]: Per-core clock enable status mask.
0x38	REG_THERMAL_IRQ	bits[4:0]: Per-core thermal interrupt pending mask.
0x50	REG_MIG_CMD	bit[0]: Start migration, bit[1]: Clear halt, bits[6:4]: Src, bits[10:8]: Dst.
0x54	REG_MIG_STATUS	bit[0]: Busy, bit[1]: Done, bit[2]: Error flag.
0x58	REG_MIG_CYCLES	bits[31:0]: Total elapsed cycles during last migration state transfer.
0x5C	REG_MIG_COUNT	bits[31:0]: Total cumulative migration count since reset.
0x60	REG_MIG_PAUSE_MASK	bits[4:0]: Active core pause mask during migration.
0x64	REG_MIG_HALT_MASK	bits[4:0]: Halted source core mask.
0x68	REG_WARM_CYCLES	bits[31:0]: Elapsed cycles during cache pre-warming phase.
0x6C	REG_WARM_PREFETCH	bits[31:0]: Number of lines successfully prefetched by warmer.
0x70	REG_EVAL_CTRL	bits[1:0]: Evaluation mode select, bit[2]: Metric freeze enable.
0x74	REG_EVAL_CYCLES	bits[31:0]: Total execution benchmark cycles.
0x78	REG_EVAL_RET_LO	bits[31:0]: Cumulative retired instructions (lower 32 bits).
0x7C	REG_EVAL_RET_HI	bits[31:0]: Cumulative retired instructions (upper 32 bits).
0x80	REG_EVAL_TMAX	bits[31:0]: Peak junction temperature recorded across all cores (Q16.16).
0x84	REG_EVAL_THROTTLE	bits[31:0]: Total clock cycles spent in throttled duty cycles.

MMIO Offset	Register Symbol	Bitfield Definitions and Architectural Role
0x88	REG_EVAL_MIG_COUNT	bits[31:0]: Total migrations executed under evaluation framework.
0x8C	REG_EVAL_PWR_GATED	bits[31:0]: Total cumulative cycles spent in power-gated state.
0x90	REG_EVAL_VIOLATIONS	bits[31:0]: Thermal violation cycles ($T_{\text{meas}} > T_{\text{warn}}$).
0x94	REG_EVAL_ENERGY	bits[31:0]: Cumulative energy proxy metric integral.
0x98	REG_EVAL_DEC_COST	bits[31:0]: Total policy-estimated decision cost.
0x9C	REG_EVAL_DEC_BEN	bits[31:0]: Total policy-estimated decision benefit score.
0xC4--0xD4	REG_MIG_HIST	bits[31:0]: Per-core cumulative migration event histograms.
0xD8--0xF4	REG_HIST_RING	8 32-bit words recording the 4 most recent 64-bit migration entries.

6.3 Peripherals: Timer, UART, and Seven-Segment Driver

- **64-Bit Timer (rtl/peripherals/Timer.sv):** Increments a 64-bit counter `mtime` on every clock cycle. Compares against `mtimecmp`; asserts `timer_irq` when `mtime` $\geq$ `mtimecmp`.
- **UART Controller (rtl/peripherals/UART.sv):** Implements standard 8-N-1 serial framing at 115200 baud (clock divisor = 98 MHz/115200 $\approx$ 851 cycles/bit). Transmits hexadecimal metric reports at program termination.
- **Seven-Segment Driver (rtl/peripherals/Seven_Seg_Driver.sv):** Multiplexes an 8-digit common-anode display at 95 Hz refresh rate. Displays real-time hexadecimal temperature of the switch-selected core (`sw[1:0]`), as well as special diagnostic error codes on hang conditions.

7 Verification Suite, Synthesis, and Physical FPGA Implementation

7.1 Automated Simulation Regression Matrix

The simulation verification environment uses Icarus Verilog and is automated by the following PowerShell script:

```
scripts/run_phase7_regression.ps1
```

7.2 FPGA Physical Implementation on AMD Xilinx UltraScale+

The complete SoC is synthesized, placed, and routed targeting the AMD Xilinx UltraScale+ XCU250 FPGA (`xcu250-figd2104-2L-e`) using Vivado batch scripts (`scripts/create_xcu250_project.tcl`):

7.3 Clock-Enable Domain Safety and Glitch Elimination

A critical engineering achievement of Prolepsis is that all duty-cycle throttling (100%, 75%, 50%, 25%) is enforced through **registered clock-enable nets** (`core_clk_en[i]`) driving the clock-enable pins of pipeline flip-flops. The primary clock net operates continuously at a pristine, unchopped 98 MHz from the global FPGA clock tree (BUFG), completely avoiding clock glitches, skew violations, and duty-cycle degradation.

#	Test Name	Target RTL Module	Pass Criteria and Verified Mechanism
1	tb_data_cache	Data_Cache.sv	Hits, misses, write-through stores, snoop invalidation, byte enables.
2	tb_branch_predictor	Branch_Predictor.sv	2-bit BHT saturation, BTB target matching, RAS call/return stack.
3	tb_migration_controller	Migration_Controller.sv	GPR $x_1 \dots x_{31}$ copy, CSR transfer, PC restore, source halt.
4	tb_migration_policy	Migration_Policy.sv	Headroom evaluation, benefit vs. cost calculation, cooldown timers.
5	tb_power_gate_controller	Power_Gate_Controller.sv	512/1024/2048 idle thresholds, save/restore handshake, wake-up.
6	tb_cache_warmer	Cache_Warmer.sv	Valid-tag sniffing, line prefetch address generation, 256-cycle bound.
7	tb_victim_buffer	Victim_Buffer.sv	Conflict line insertion, fully associative hit swap, FIFO eviction.
8	tb_stride_prefetcher	Stride_Prefetcher.sv	Stride detection, confidence increment, speculative line prefetch.
9	tb_rvc_decompressor	RVC_Decompressor.sv	Quadrant 0/1/2 16-bit expansion into standard 32-bit RISC-V opcodes.
10	tb_hazard_unit	Hazard_Unit.sv	EX/MEM operand forwarding, load-use stalls, branch flush bubbles.
11	tb_qos_arbiter	qos_arbiter.sv	3-bit priority grant, 200-cycle age starvation boost, round-robin.
12	tb_eval_framework	Eval_Framework.sv	Metric accumulation, freeze register assertion, UART hex report.
13	tb_csr_unit	CSR_Unit.sv	Machine CSR read/write, exception trap entry (mepc), MRET.
14	tb_pipelined_alu	Pipelined_ALU.sv	Arithmetic, shifts, comparisons, DSP multiplication, multi-cycle division.
15	tb_rv32_core	RV32_Core.sv	Complete 5-stage core execution of arithmetic and branch instruction streams.
16	tb_instruction_cache	Instruction_Cache.sv	Single-cycle hit delivery, 4-word burst line fills, tag replacement.
17	tb_multicore	Orionrv.sv	5-core concurrent execution, snoop broadcast coherence, bus arbitration.
18	tb_kalman_filter	Kalman_Predictor.sv	Multiplier pipeline, power-of-two shift gain, temperature prediction.
19	tb_phase_detector	Phase_Detector.sv	4096-cycle window sampling, dual EMA smoothing, 8-sample hysteresis.
20	tb_orionrv_matched_ceiling	Orionrv.sv	Closed-loop thermal comparison: Mode 0 vs. Mode 1 vs. Mode 2 vs. Mode 3.
21	tb_orionrv_integration	Orionrv.sv	Top-level SoC boot from ROM, MMIO read/write, D-pad mode transitions.

Table 7: Comprehensive 21-Test Simulation Regression Matrix (All Passing 21/21).

FPGA Resource Category	Utilized Count	Device Capacity	Utilization Percentage
Lookup Tables (LUT as Logic)	142,102	1,341,120	10.60%
Flip-Flops / Registers (FF)	87,579	2,682,240	3.26%
DSP48 Arithmetic Slices	184	12,288	1.50%
Block RAM (BRAM18 Blocks)	32	5,376	0.60%
Target Clock Frequency	98.00 MHz ($T_{\text{clk}} = 10.204 \text{ ns}$)		
Worst Negative Slack (WNS)	+0.057 ns (Timing Closed)		
Worst Hold Slack (WHS)	+0.011 ns (Timing Closed)		
Total On-Chip Power	3.720 W		

Table 8: Post-Implementation Resource Utilization and Timing Report on XCU250.

8 Architectural Trade-Off Analysis and Comparative Evaluation

Architectural Decision	Engineering Rationale	Design Trade-Off & Mitigation
RV32IMAC ISA	32-bit datapaths provide optimal balance of computational capability and low FPGA resource footprint.	Lower address space than RV64; fully sufficient for embedded 256 KB SRAM scale.
5-Stage In-Order	Deterministic instruction latency, simple stall logic, minimal power dissipation.	Lower IPC than out-of-order execution; compensated by branch predictor, victim buffers, and prefetchers.
Heterogeneous (4P+1E)	Demonstrates asymmetric scheduling and power conservation during light workloads.	Requires asymmetric default QoS configuration; handled cleanly by package parameters.
Direct-Mapped L1	Single-cycle access latency, zero multiplexer delay, optimal block RAM inference.	Vulnerable to conflict misses; mitigated via 2-entry victim buffers and stride prefetchers.
Write-Through Caches	Main memory is perpetually coherent; eliminates dirty-line eviction and complex writeback buffers.	Generates more bus write traffic; mitigated by single-entry store buffer write-merging.
Single-Word AXI Bus	Bounded, deadlock-free single-outstanding transactions; simple, verifiable FSMs.	Lower burst bandwidth than a crossbar; mitigated by QoS priority arbitration.
Q16.16 Fixed Point	High mathematical precision for differential thermal integration without floating-point FPU area.	Limited dynamic range; managed via strict bitfield normalization shifts.
Division-Free Kalman	Power-of-two shift gain adaptation eliminates 120 ns divider critical paths.	Minor theoretical approximation in gain optimality; maintains full stability in practice.
Dual EMA + Hysteresis	Filters high-frequency phase noise using simple bit-shift weighting ($\alpha = 1/8, 1/64$).	Introduces 8-sample response lag; protects system from oscillatory control chatter.
Cache Pre-Warmer	Pre-populates destination L1 cache with active source working lines during task migration.	Consumes bus bandwidth during migration pause; strictly bounded to 256 clock cycles.

9 Exhaustive Technical Question and Answer Reference

1. What is the fundamental contribution of the Prolepsis architecture?

Prolepsis demonstrates that multicore processor temperature can be managed proactively in hardware before thermal thresholds are breached. By integrating execution telemetry, a fixed-point Kalman temperature estimator, workload phase detection, dynamic bus QoS scheduling, proactive task migration, and cache pre-warming, Prolepsis prevents thermal overshoot while preserving workload throughput.

2. Why does reactive thermal throttling fail in multicore systems?

Reactive thermal management acts only after a measured thermal threshold is crossed. Because the silicon die and heat spreader possess significant thermal capacitance, thermal energy continues to conduct into the junction even after power is reduced, causing thermal overshoot above critical limits and triggering aggressive clock chopping.

3. Why is workload phase classification essential for thermal control?

Compute-bound workloads generate high dynamic switching activity and heat; throttling them yields substantial power and thermal reduction. Conversely, memory-bound workloads stall on external memory and generate very low dynamic heat; throttling a memory-bound core penalizes execution throughput without providing thermal benefit.

4. How does Prolepsis perform clock throttling safely on an FPGA?

Throttling is implemented exclusively via registered clock-enable signals (`clk_en`) driving the clock-enable inputs of pipeline registers (yielding 100%, 75%, 50%, or 25% duty cycles). The primary FPGA clock net runs at an uninterrupted, glitch-free 98 MHz on dedicated global clock routing buffers (BUFG).

5. How is architectural correctness guaranteed during task migration?

Task migration is executed by a hardware finite state machine:

1. Both source and destination pipelines are synchronously paused.
2. General-purpose registers ($x_1 \dots x_{31}$) and machine CSRs are copied over dedicated point-to-point migration buses.
3. The cache warmer pre-populates destination instruction cache lines.
4. The destination PC is restored from the source checkpoint, and the source core is halted.

6. How does the cache warmer improve post-migration performance?

When execution migrates to a cold destination core, the pipeline normally stalls on consecutive L1 instruction cache misses. The cache warmer sniffs valid tags from the source I-cache and issues line-aligned prefetch requests to pre-fill the destination I-cache during the migration window, eliminating cold-start stall bursts.

7. How is multicore cache coherence maintained?

Because all L1 data caches are write-through, all stores update main memory immediately. The top-level fabric broadcasts the store address across an on-chip snoop bus. Remote peer caches invalidate matching cache lines and clear atomic reservations, providing lightweight coherence without directory overhead.

8. How does the QoS arbiter prevent low-priority master starvation?

The QoS arbiter maintains an 8-bit age counter for each master. If a valid request waits for 200 consecutive cycles without receiving a grant, its effective priority is boosted with a high-order bit, guaranteeing bounded access latency for lower-priority traffic.

9. What are the active thermal thresholds in the Prolepsis design?

In signed Q16.16 format:

- Ambient Temperature (T_{amb}): 25.0°C (0x0019_0000)
- Warning Threshold (T_{warn}): 85.0°C (0x0055_0000)
- Layer 1 Safety Ceiling (T_{cap}): 87.0°C (0x0057_0000)
- Critical Threshold (T_{crit}): 90.0°C (0x005A_0000)

10. What are the known architectural boundaries and simplifications in the current RTL?

The active RTL is a fully verified hardware prototype with clearly defined research boundaries:

- Thermal sensing uses a synthesizable 3-node RC model rather than external analog XADC sensors.
- The interconnect is a single-outstanding transaction AXI4-Lite arbiter rather than a full crossbar or Network-on-Chip (NoC).
- VGA and MAC accelerator master slots serve as mapped endpoints and stubs.

10 Repository Manifest and Automation Quick Reference

10.1 Essential Source Files for Architectural Review

- SoC Top-Level Integration: rtl/Orionrv.sv
- RISC-V Core Pipeline: rtl/core/RV32_Core.sv
- Kalman Predictor Engine: rtl/thermal/Kalman_Predictor.sv
- Workload Phase Detector: rtl/thermal/Phase_Detector.sv
- Hierarchical Thermal Controller: rtl/thermal/Thermal_Controller.sv
- Task Migration Engine: rtl/thermal/Migration_Policy.sv,
rtl/thermal/Migration_Controller.sv
- Instruction Cache Warmer: rtl/thermal/Cache_Warmer.sv
- QoS Bus Interconnect: rtl/bus/axi_interconnect.sv, rtl/bus/qos_arbiter.sv

10.2 Primary Verification and Build Commands

```
# 1. Execute complete 21-test automated simulation regression suite
powershell -ExecutionPolicy Bypass -File scripts/run_phase7_regression.ps1

# 2. Re-generate boot ROM hex payload from sw/boot.S
powershell -ExecutionPolicy Bypass -File scripts/gen_boot_hex.ps1

# 3. Create Vivado project and synthesize/implement for AMD Xilinx XCU250
vivado -mode batch -source scripts/create_xcu250_project.tcl
```